



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: ARQUITECTURA DE COMPUTADORAS

INTERFAZ GRÁFICA DE QTSPIM

Alumno:

Areli Alejandra Guevara Badillo

Grupo 5CM2

Prof. Gelacio Castillo Cabrera

martes, 4 de marzo de 2025

INTRODUCCIÓN

QtSPIM es un simulador gráfico de la arquitectura MIPS (Microprocessor without Interlocked Pipeline Stages) que permite ejecutar y depurar programas escritos en lenguaje ensamblador MIPS. Es una versión con interfaz gráfica de SPIM, desarrollado originalmente por James Larus, y está diseñado principalmente para fines educativos, facilitando la comprensión de la arquitectura MIPS sin necesidad de hardware físico.

El simulador proporciona herramientas visuales para examinar registros, memoria y el flujo de ejecución del programa, lo que lo convierte en una herramienta clave para estudiantes, profesores e investigadores en el campo de la arquitectura de computadores.

En este documento se describe la interfaz gráfica del simulador QtSPIM, explicando la función y organización de cada uno de sus componentes. Se detallan los diferentes paneles que conforman la interfaz, así como su utilidad dentro del simulador. También se presentan las herramientas disponibles para la ejecución y depuración de programas en ensamblador MIPS. Además, se aborda la estructura de la memoria en QtSPIM, incluyendo los segmentos de texto, datos y pila.

CONTENIDO

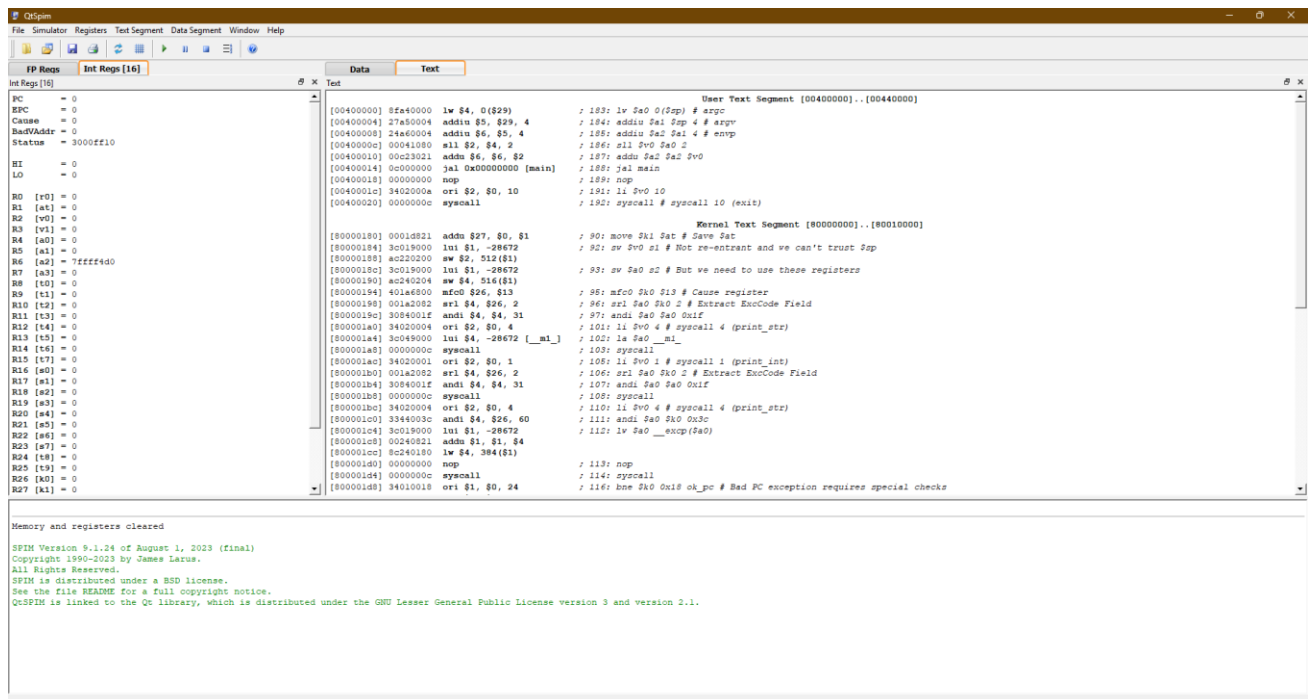
¿Qué es MIPS y por qué es importante?

MIPS es una arquitectura de procesador basada en un conjunto reducido de instrucciones (**RISC - Reduced Instruction Set Computing**), ampliamente utilizada en sistemas embebidos, dispositivos de red y computación de alto rendimiento. La simulación de programas en MIPS permite a los estudiantes comprender:

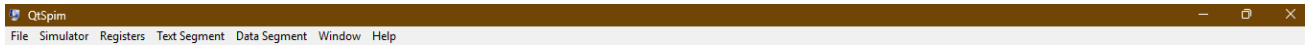
- El funcionamiento de un procesador basado en registros.
- La interacción entre memoria, CPU y periféricos.
- La ejecución secuencial de instrucciones y su impacto en el estado del sistema.

Estructura de la Interfaz de QtSPIM

La interfaz gráfica de QtSPIM está diseñada para dividir las funciones del simulador en paneles específicos que organizan la información de forma clara.



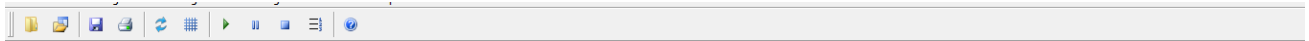
1. Barra de Menú



Ubicada en la parte superior de la ventana, la barra de menú agrupa las principales opciones para la ejecución y gestión del simulador:

- **File (Archivo):**
 - Abrir programas en ensamblador MIPS (.s).
 - Guardar cambios en el código fuente.
 - Cerrar archivos y salir del simulador.
- **Simulator (Simulador):**
 - Cargar y ejecutar programas.
 - Restablecer registros y memoria.
 - Configurar la ejecución paso a paso.
- **Registers (Registros):**
 - Mostrar y modificar registros de propósito general.
 - Visualizar registros en coma flotante y especiales (HI, LO, PC).
- **Text Segment (Segmento de Texto):**
 - Visualizar y modificar el código ensamblador cargado en memoria.
- **Data Segment (Segmento de Datos):**
 - Explorar los datos almacenados en la memoria del programa.
- **Window (Ventana):**
 - Administrar la distribución de los paneles dentro de la interfaz gráfica.

2. Barra de Herramientas



Situada justo debajo de la barra de menú, contiene accesos directos a las funciones más utilizadas del simulador. Incluye botones para:

- **Abrir un archivo en ensamblador (.s).**
- **Ejecutar el código** cargado en la memoria del simulador.
- **Pausar o detener la ejecución** del programa.
- **Ejecutar el programa instrucción por instrucción.**
- **Restablecer registros y memoria** para probar nuevamente un código.

3. Panel de Registros

En este panel se muestra el estado actual del conjunto de registros de la arquitectura MIPS. Está dividido en tres secciones principales:

Registros de Propósito General

Los registros de propósito general son **32 registros (R0 a R31)** utilizados para almacenar datos y direcciones temporales durante la ejecución de un programa. Se pueden clasificar en:

- **Registros reservados:** \$zero (R0), que siempre contiene el valor 0.
- **Registros para argumentos de funciones:** \$a0 - \$a3.
- **Registros para valores de retorno:** \$v0 - \$v1.

- Registros temporales: \$t0 - \$t9.
- Registros guardados: \$s0 - \$s7.
- Registros especiales: \$sp (Stack Pointer), \$fp (Frame Pointer), \$ra (Return Address).

FP Regs		Int Regs [16]
Int Regs [16]		
PC	=	0
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	0
R0	[r0]	= 0
R1	[at]	= 0
R2	[v0]	= 0
R3	[v1]	= 0
R4	[a0]	= 0
R5	[a1]	= 0
R6	[a2]	= 7ffff4d0
R7	[a3]	= 0
R8	[t0]	= 0
R9	[t1]	= 0
R10	[t2]	= 0
R11	[t3]	= 0
R12	[t4]	= 0
R13	[t5]	= 0
R14	[t6]	= 0
R15	[t7]	= 0
R16	[s0]	= 0
R17	[s1]	= 0
R18	[s2]	= 0
R19	[s3]	= 0
R20	[s4]	= 0
R21	[s5]	= 0
R22	[s6]	= 0

Registros Especiales

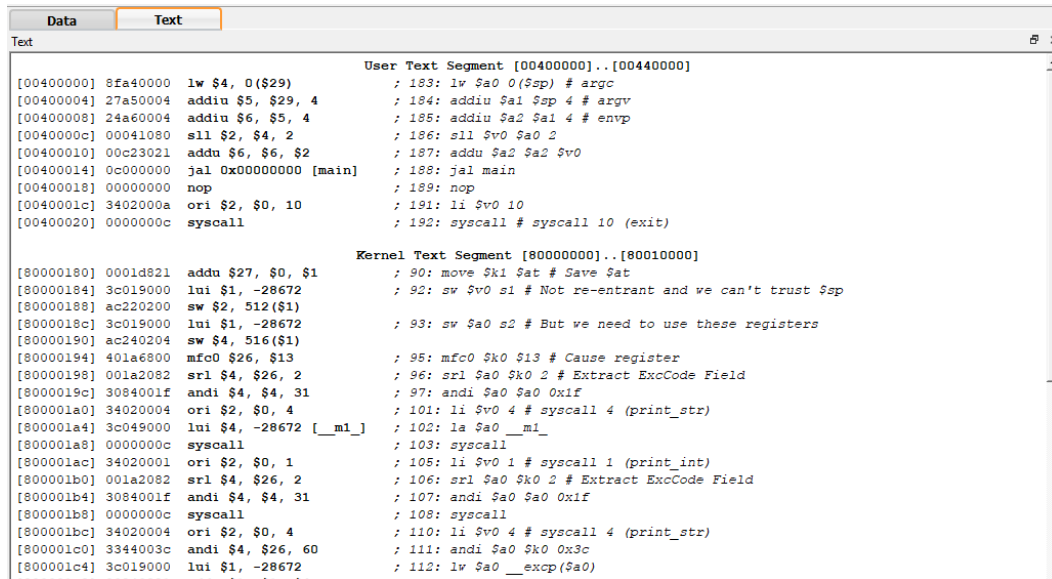
Incluyen el **contador de programa (PC)**, que almacena la dirección de la siguiente instrucción a ejecutar, y los registros **HI** y **LO**, utilizados para almacenar el resultado de operaciones de multiplicación y división.

Registros de Punto Flotante

Cuando se requiere trabajar con **números en coma flotante**, MIPS proporciona **32 registros de punto flotante (FP0 a FP31)** que permiten cálculos en **precisión simple y doble**.

FP Regs		Int Regs [16]
Int Regs [16]		
PC	=	0
EPC	=	0
Cause	=	0
BadVAddr	=	0
Status	=	3000ff10
HI	=	0
LO	=	0
R0	[r0]	= 0
R1	[at]	= 0
R2	[v0]	= 0
R3	[v1]	= 0
R4	[a0]	= 0
R5	[a1]	= 0
R6	[a2]	= 7ffff4d0
R7	[a3]	= 0
R8	[t0]	= 0
R9	[t1]	= 0
R10	[t2]	= 0
R11	[t3]	= 0
R12	[t4]	= 0
R13	[t5]	= 0
R14	[t6]	= 0
R15	[t7]	= 0
R16	[s0]	= 0
R17	[s1]	= 0
R18	[s2]	= 0
R19	[s3]	= 0
R20	[s4]	= 0
R21	[s5]	= 0
R22	[s6]	= 0

4. Panel del Segmento de Texto (Código del Programa)



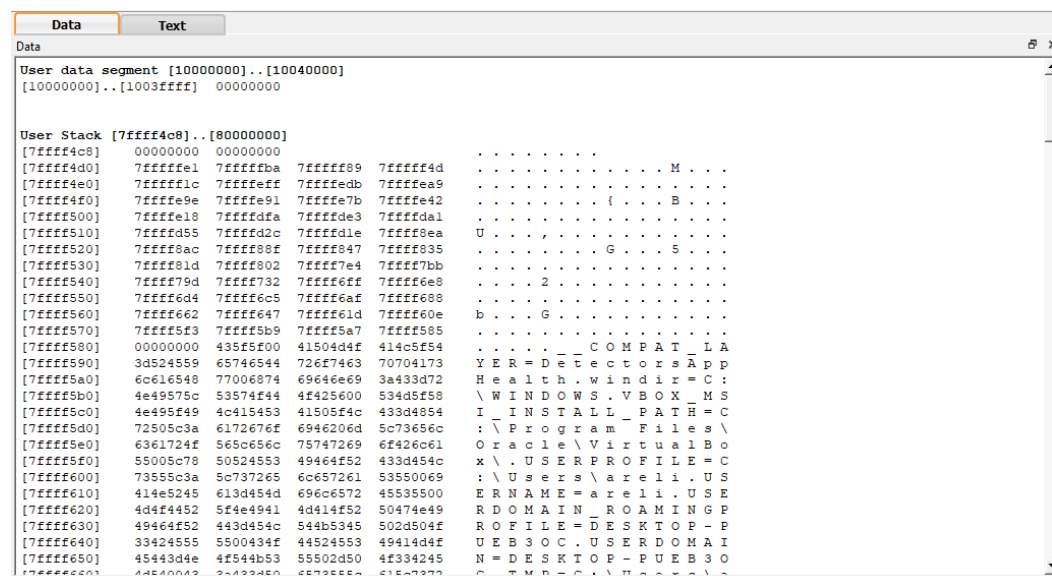
The screenshot shows the 'Text' panel of a debugger. It displays assembly code for two segments: 'User Text Segment' and 'Kernel Text Segment'. The 'User Text Segment' starts at address 00400000 and contains instructions like 'lw \$4, 0(\$29)', 'addiu \$5, \$29, 4', 'addiu \$6, \$5, 4', 'sll \$2, \$4, 2', 'addu \$6, \$6, \$2', 'jal 0x00000000 [main]', 'nop', 'ori \$2, \$0, 10', and 'syscall'. The 'Kernel Text Segment' starts at address 80000000 and contains instructions like 'addu \$27, \$0, \$1', 'lui \$1, -28672', 'sw \$2, 512(\$1)', 'lui \$1, -28672', 'sw \$4, 516(\$1)', 'mfc0 \$26, \$13', 'srl \$4, \$26, 2', 'andi \$4, \$4, 31', 'ori \$2, \$0, 4', 'lui \$4, -28672 [__m1_]', 'syscall', 'ori \$2, \$0, 1', 'srl \$4, \$26, 2', 'andi \$4, \$4, 31', 'syscall', 'ori \$2, \$0, 4', 'andi \$4, \$26, 60', and 'lui \$1, -28672'. Comments on the right side of the instructions provide context for the code.

Este panel muestra el código en ensamblador almacenado en la memoria, identificado por la directiva **.text**. Está dividido en dos partes:

- **User Text Segment:** Contiene el código ensamblador escrito por el usuario.
- **Kernel Text Segment:** Contiene instrucciones del kernel, utilizadas para manejar **excepciones, interrupciones y llamadas al sistema**.

Cada instrucción en el segmento de texto está identificada por una dirección de memoria, que comienza en 0x00400000.

5. Panel del Segmento de Datos y Pila



The screenshot shows the 'Data' panel of a debugger. It displays memory contents for two segments: 'User data segment' and 'User Stack'. The 'User data segment' starts at address 10000000 and contains the value 00000000. The 'User Stack' starts at address 7ffff4c8 and contains a series of memory addresses and values, including 00000000, 7ffff4c8, 7ffff4d0, 7ffff4e0, 7ffff4f0, 7ffff500, 7ffff510, 7ffff520, 7ffff530, 7ffff540, 7ffff550, 7ffff560, 7ffff570, 7ffff580, 7ffff590, 7ffff5a0, 7ffff5b0, 7ffff5c0, 7ffff5d0, 7ffff5e0, 7ffff5f0, 7ffff600, 7ffff610, 7ffff620, 7ffff630, 7ffff640, and 7ffff650. The values are displayed in hexadecimal and include comments like 'COMPAT_LA', 'YER=DetectorsApp', 'Heal th. windir=C:', 'WINDOWS.VBOX_MS', 'INSTALLED_PATH=C:', 'Program Files\Oracle\VirtualBox', 'Users\arelii.US', 'ERNAME=arelii.US', 'DOMAIN_ROAMINGP', 'ROFILE=DESKTOP-P', 'UEB3OC.USERDOMAI', 'N=DESKTOP-PUEB3O', and 'CMD=C:\Program Files\'. The panel also shows the 'Data' tab selected at the top.

Muestra los datos almacenados en memoria, divididos en:

- **Segmento de Datos (.data):** Almacena variables definidas en ensamblador MIPS, comenzando en 0x10000000.
- **Segmento de Pila (stack):** Utilizado para gestionar llamadas a funciones y almacenamiento temporal, con una dirección inicial de 0x7ffffeff.

6. Consola de Entrada y Salida



Simula la interacción del usuario con el programa, mostrando la salida generada y permitiendo ingresar valores cuando el código lo requiere.

7. Panel de Depuración



Muestra información de control y errores durante la ejecución del código ensamblador. Es fundamental para detectar problemas y corregirlos de manera efectiva.

Ejemplo de Ejecución del Archivo HelloWorld.s (incluido al descargar QtSPIM)

Este código en ensamblador MIPS imprime "Hello World" en la consola y luego intenta cargar un valor de memoria desde una variable externa llamada foobar. Finalmente, el programa regresa al llamador usando jr \$ra.

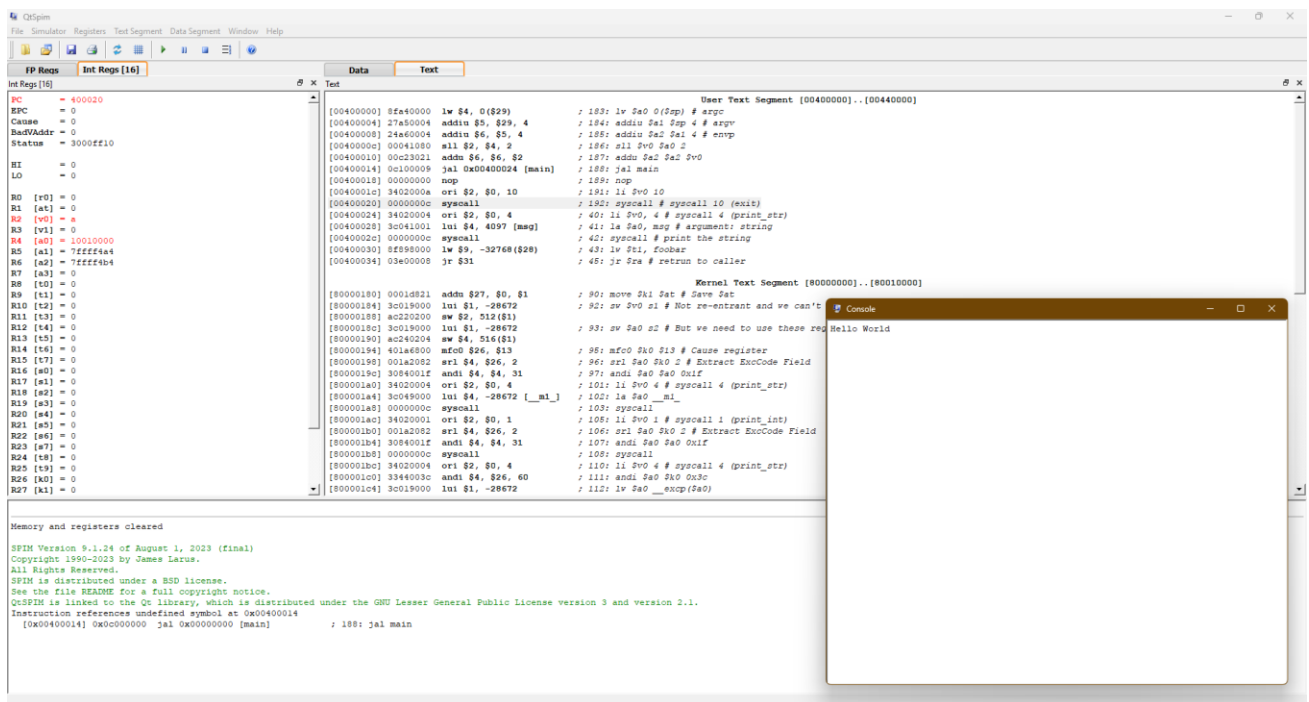
```
msg:      .data                # Inicio del segmento de datos
         .ascii "Hello World" # Define la cadena de texto "Hello World" con terminador nulo (\0)
         .extern foobar 4      # Declara 'foobar' como una variable externa con un tamaño de 4 bytes

         .text                # Inicio del segmento de código
         .globl main          # Declara 'main' como una etiqueta global (punto de entrada del programa)

main:
         li $v0, 4            # Carga el código de servicio 4 en $v0 (syscall para imprimir cadenas)
         la $a0, msg          # Carga la dirección de 'msg' en $a0 (argumento de la syscall)
         syscall              # Llamada al sistema para imprimir la cadena en la consola

         lw $t1, foobar        # Carga en $t1 el valor almacenado en 'foobar' (debe estar definido en otro
                                # archivo o memoria)

         jr $ra               # Retorna al llamador (finaliza el programa si 'main' es el punto de entrada)
```



REFERENCIAS

- QtSpim Tutorial — ECS Networking. (n.d.). Recuperado de <https://ecs-network.serv.pacific.edu/past-courses/2014-fall-ecpe-170/tutorials/qtspim-tutorial>
- Cmput. (n.d.). GitHub - cmput229/Lab_Intro: CMPUT 229 Lab 1 - The SPIM/QTSPIM Environment. Recuperado de https://github.com/cmput229/Lab_Intro
- Ed Jorgensen (Junio, 2016) MIPS Assembly Language Programming using QtSpim. Recuperado de https://class.ece.iastate.edu/cpre381/resources/programming_with_QtSpim.pdf